MODULE 02 / TEAMMATE FIELD GUIDE

Resume Parsing

Read a candidate's resume and turn its supported statements into editable skill claims and background fields. Keep each claim attached to its source, then hand the reviewed extraction to Profile Builder.

JD | **Resume** | Profile | Talent Check | Match + GPS

The shared journey; each module also runs independently.

You receive	You return
A PDF or DOCX resume, plus optional explicit consent for Groq to process this input's text.	A saved Extraction with source_type=resume: name/email, skills, education, projects, experience and other supported sections. It is not a proficiency assessment.

Your responsibility

Participant Brief p.7 labels this LLM EXTRACTION. Preserve PDF/DOCX support, structured skills, reliable background fields and tolerance for varied layouts. Source evidence and explicit review are ProofPath additions.

A successful handoff looks like this

Two contrasting resumes produce different source-backed claims. The reviewer can correct a name/section, save it, and import it without any automatic competency rating.

> **Start from the working baseline**
>
> Assume YOU own this module. Run and understand the supplied working source, finish or improve your part, and test it. Push the verified working copy to YOUR OWN GitHub repository. Return its URL, commit SHA, actual test results and known limitations to the Scrum Master. No central merge or PR is required from you.

Before your module	After your module
The candidate supplies a resume. You do not need a JD, company or benchmark to parse it.	Module 3 receives a reviewed resume ID. Its import-resume operation merges the saved Extraction into a profile draft; it does not copy values from your UI.

Read in order: 2 build plan | 3 screen layout | 4 API | 5 logic | 6 integration | 7 tests | 8 presentation | 9 exact field reference

02 / IMPLEMENTATION PLAN

What to do, in order.

Keep changes inside your feature folder. Each step has an observable result; do not begin by replacing the whole app.

01 Trace one resume

Read the Mira synthetic CV and your service/router/UI. Identify exactly what is extracted and where Profile Builder reads it.

02 Reuse ingestion

Use the shared PDF/DOCX reader and private document storage. Do not add a second upload endpoint, public CV URL or browser-only PDF parser.

03 Extract supported facts

Ask Groq for identity, explicit/inferred skill claims and background sections. Include unfamiliar technologies. Missing facts remain empty; section arrays contain original quotes.

04 Keep source grounding

Preserve original quote, PDF page or null DOCX page, source document ID and available spans. Treat document instructions as untrusted content.

05 Require review

Show every proposed skill and supported section. Let the user correct/delete/add claims and explicitly confirm unreviewed skills.

06 Save the extraction

PATCH the full current Extraction with reviewed status. Retain accepted data if a later workspace refresh fails.

07 Prove import

Give Module 3 the reviewed ID. Verify fields/evidence survive import and all twelve candidate competency levels remain null unless already set by the user.

Files to open first

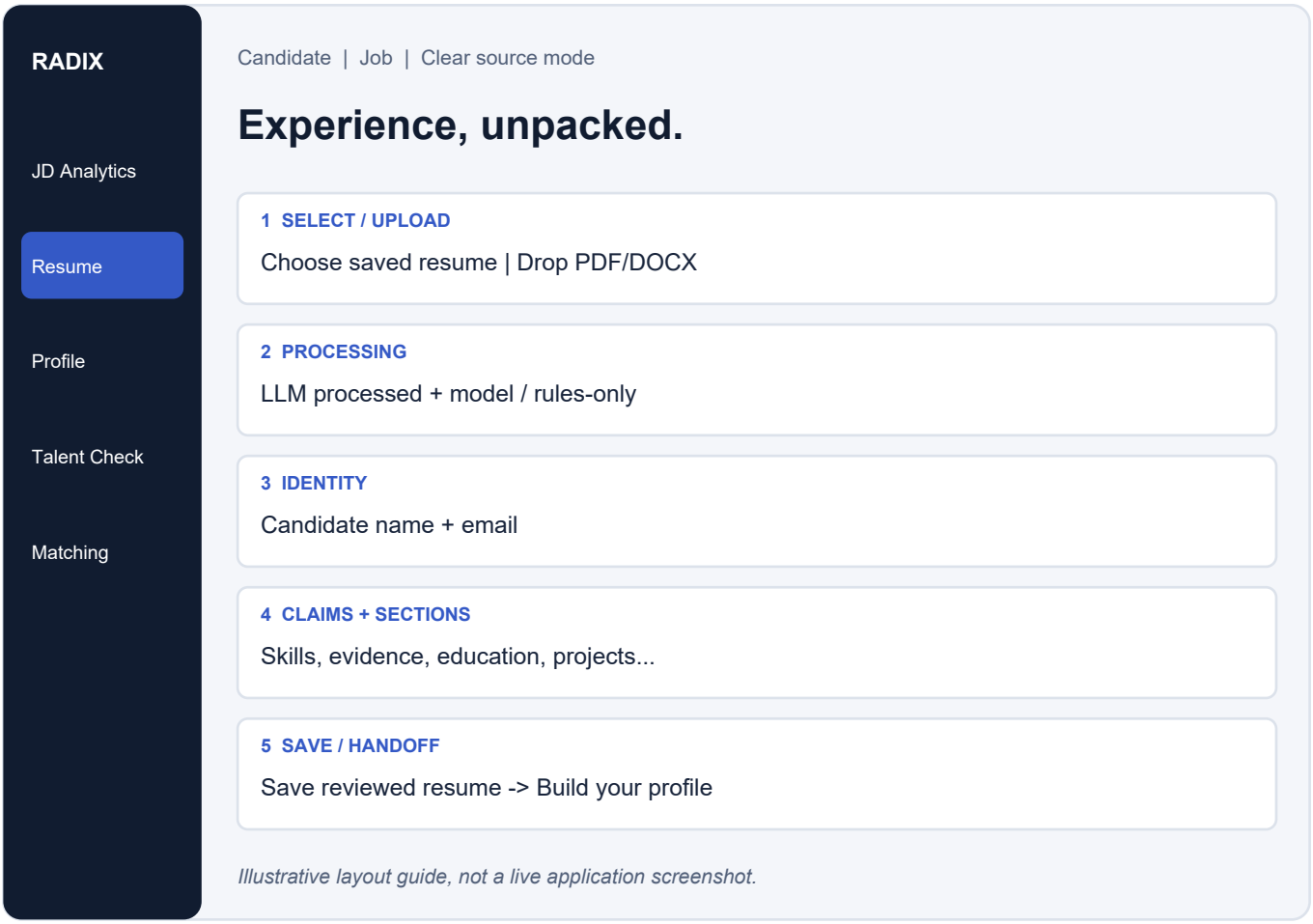
All paths below are relative to resume_parsing/.

File	What it owns
backend/service.py	Resume-specific instructions, local section rules and postprocessing.
backend/router.py	Composes the shared extraction HTTP plumbing for resumes.
frontend/ResumeParsingPage.tsx	Upload, saved-resume selection, field editing and review save.
tests/test_extraction.py + test_audit.py	Resume evidence, layout and section regressions.

03 / SCREEN AND USER EXPERIENCE

What it should look like.

Keep the current app shell, navigation, typography and shared controls. The drawing shows information order, not a redesign request.



Screen state	Expected visible behavior
No resume	Explain accepted formats and human review.
Wrong/incomplete extraction	Let the reviewer fix it; never hide the source.
Groq unavailable	Show rules-only fallback and likely omissions; preserve the input.
Empty/scan/encrypted PDF	Explain that readable text is required; OCR is not implemented.

Visual rules: 16 px body/form text, 14 px minimum supporting labels; clear headings; 44 px touch targets. Navy #111C30, blue #3459C6, white cards and #F4F6FA canvas. Use the existing Inter web font. On mobile, stack panels and keep actions readable. Do not rely on color alone for status.

04 / EXACT API BOUNDARY

Input, output and format.

Use the current Pydantic/OpenAPI contract. A UI label can change; an agreed JSON field cannot be renamed independently.

Endpoint	Body or result
POST /api/resumes/extract	Multipart: file; hosted_consent=false unless opted in.
GET /api/resumes[/ID]	List/load owned saved extractions.
PATCH /api/resumes/ID	Full Extraction + current revision + reviewed status.
DELETE /api/resumes/ID	Delete extraction; delete original separately in Data & privacy.

Upload fields; multipart, not a JSON string containing a path

```
file = actual PDF/DOCX bytes
hosted_consent = "false"

POST /api/resumes/extract
Header: X-ProofPath-Client: web
Content-Type: browser sets multipart boundary
```

Current synthetic response excerpt

Only selected fields are shown below. Page 9 describes the complete shape. Snapshot UUIDs are teaching examples, not IDs to reuse in a running app.

```
{
  "id": "841baae1-c160-5194-98d3-1cc98db72fb7",
  "revision": 2,
  "source_type": "resume",
  "source_file": "mira-cv-SYNTHETIC.pdf",
  "name": "Mira Sen (synthetic)",
  "email": "mira@example.test",
  "education": [
    "Synthetic Institute - BSc Computing (invented qualification)"
  ],
  "internships": [
    "Synthetic training placement - maintained a small reporting service."
  ],
  "review_status": "reviewed",
  "llm_status": "rules_only",
  "document_id": "cbe3412c-d457-56de-98bd-67f54cb8636e"
}
```

Every mutation needs X-ProofPath-Client: web (or cli). Cloud requests also need the current user's Bearer JWT. The shared web helper supplies these headers. File upload uses multipart; never manually set its boundary. JSON fields use snake_case.

05 / REASONING AND BUSINESS RULES

A mention is a claim. It is not a measured skill level.

A resume is evidence of what the document says. It does not establish that the candidate is skilled, truthful or independently verified. Preserve that distinction from extraction to the final presentation.

Situation	Exact expected reasoning
'Built a Python pipeline'	Extract Python with the exact project quote. Confidence describes how clearly the text supports that extraction.
'Rust' is not in the old vocabulary	A source-grounded Groq proposal can preserve it. Use a sensible RADIX category; do not drop it because a fixed dictionary lacks it.
'Prepared messy records for analysis'	May support an inferred Data Cleaning proposal. Mark inferred; a human must judge the meaning, even when the quote matches.
'No production Kubernetes experience'	Do not turn it into positive competency. Keep a negated/conflicting claim unresolved and source-visible.
Missing name, date or qualification	Leave blank/empty instead of completing it from guesses. Do not merge two employers into one experience entry.

Important edge case

PDF pages start at 1. DOCX has no reliable physical page count here, so source_page=null. Multi-column reconstruction is conservative and emits review warnings. Scanned PDFs do not have OCR support.

The LLM path

Document reader -> rules as hints -> consented Groq strict JSON -> schema/quote checks -> server-owned IDs -> editable proposals -> reviewed save. Model claims start needs_review; verification stays unverified. One bounded regeneration is allowed for invalid output. Provider errors produce an honest fallback by default.

Never cross this boundary

Never assign 1-10 ratings from resume mentions, confidence, project count or an LLM. Imported claims remain unverified. Private CV processing on Groq requires the input's explicit consent.

06 / INTEGRATION AGREEMENT

Own it. Test it. Publish your part.

Your task is a working module on your own GitHub. Keep the supplied shared contracts and exact output formats so the Scrum Master can integrate the returned code.

Team rule	What you do
Service boundary	backend/service.py: extract(sections, filename, hosted=False) -> Extraction. Keep resume source_type and the reviewed Profile Builder handoff.
One feature owner	Own resume_parsing/backend, frontend, tests and docs. Keep the common support area in your clone. Run your module independently before claiming it works.
One contract source	Read shared/contracts/models.py. OpenAPI and generated TypeScript follow it. Do not maintain a second handwritten schema or change categories alone.
One storage/auth path	Use shared store/document/auth helpers. Do not add a second database, anonymous cloud owner, public CV URL or frontend API secret.
One current record	Use actual returned IDs/revisions, owned by the signed-in user. Pass saved IDs to consumers; never hardcode teaching snapshot IDs.
Your GitHub	Create a private repository on YOUR account. Rename the old origin to scrum-master, set your new origin, review staged files and scan for secrets, then commit and push. See SUBMIT_YOUR_REPOSITORY.md.
Return working evidence	Fill DELIVERY.md with your URL, commit SHA, actual commands/results, three normal and three failure cases, synthetic screenshots and known bugs. If a check fails, fix it or report it honestly; do not copy passing counts.

The official category vocabulary

Every extracted skill has exactly one official code or OTHER. The extra descriptive skill families are filters, not replacement categories. Only the twelve official codes can have numeric ratings or company expectations.

Category	Category
COD - Coding	CLOUD - Cloud
DSA - Data Structures & Algorithms	SQL - SQL
OOD - Object-Oriented Design	SWE - Software Engineering
APTI - Aptitude	SYSD - System Design
COMM - Communication	NETW - Networking
AI - AI	OS - Operating Systems

Keep these meanings separate

Confidence = how clearly the document supports an extraction. Competency = explicit self-rating, null or 1-10. Review = a human checked the output. Verified = independently established; this app's claims remain unverified. Hypothetical = GPS preview only.

07 / RUN AND VERIFY

Your definition of done.

A module must run independently as well as inside the shared app. Record actual commands and failures; a screenshot or a copied passing count is not test evidence.

Use the existing project, not a second app

From the repository root, install the existing requirements/lockfiles if needed. Open two terminals. If the ports are already running, use them; do not kill unknown processes. Never overwrite an existing .env.

```
.\resume_parsing\run.ps1 api
# Separate terminal:
.\resume_parsing\run.ps1 web
# Real standalone API check:
.\resume_parsing\run.ps1 check
```

Clone the full repo and install root dependencies (README). Run these from its root. Open http://127.0.0.1:5173/; Load synthetic inputs supplies upstream fixtures. No key is needed for rules. A Groq key belongs only in the private root .env; restart Python after editing it.

Test	Pass means
Two contrasting resumes, PDF + DOCX	Different skill/background outputs with defensible evidence; no candidate-name shortcuts.
Unfamiliar technology + varied heading	Retain supported proposal; separate projects, internships and education correctly.
Negated/conflicting mention	Needs review; cannot automatically satisfy matching.
Long unbroken word / missing @	No pathological email-regex slowdown; no invented email.
Review/save/import	Profile draft contains the reviewed fields and source references; ratings unchanged.
Provider error / invalid quote / no consent	Bounded failure/fallback; unsupported claim rejected; no unauthorized request.

Your focused commands

```
.\.venv\Scripts\python.exe -m pytest resume_parsing/tests -q
npm run test -- resume_parsing/frontend
```

Before returning: run typecheck, lint, build and the standalone browser check. Use the project virtualenv for Python. Keep assertions meaningful and record actual results.

Error behavior to demonstrate

422 = invalid/unreadable input; 413 = size limit; 415 = unsupported file; 404 = missing/unowned record; 409 = stale revision; 503 = unavailable storage/benchmark. Groq may fall back and return a successful Extraction with llm_status=unavailable instead of an HTTP error. Show the status, not just HTTP 200.

08 / LIVE DEMO AND PRESENTATION

Show it. Explain it. Hand it over.

Suggested speaking time: four minutes per module. Five modules use about 20 minutes, leaving 10 for the problem, architecture, challenge and close within the 30-minute presentation. Q&A is separate (15 minutes).

Reproducible synthetic demo

1. Open Resume Parsing. Upload demo/final/mira-cv-SYNTHETIC.pdf with hosted consent off for the fixed rules demo.

2. Show Mira Sen (synthetic), email, six claims: Python, SQL, Pandas, NumPy, Git and Linux.

3. Open Python evidence and its original source. Show education, selected work/projects and the synthetic internship.

4. Point out that extraction confidence and verified ability are different. No numeric competency rating was produced.

5. Save reviewed resume; hand its current ID to Profile Builder. For a live Groq demo, use synthetic input and show the real processing result, including any review corrections.

What to say and show

Time	Speaking cue
0:00-0:35	My module reads the candidate's document. It extracts supported facts, not a hiring decision.
0:35-1:25	Upload Mira, show identity and six extracted claims. Open one exact project quote.
1:25-2:15	Explain explicit versus inferred claims, unfamiliar terms and why negation needs review.
2:15-3:10	Show editable projects/education. Explain format limits and LLM/rules status honestly.
3:10-4:00	Save the review, explain the resume-ID handoff and emphasize that ratings remain unknown.

Transition to the next speaker

We now have reviewed resume evidence. Module 3 adds the rest of the candidate's story and saves a complete profile.

Use this language honestly

Say which parts you implemented, adapted, tested or learned. If showing the working team baseline, call it the integrated team implementation. Do not claim a live model, official benchmark or feature history that has not actually been demonstrated.

09 / FIELD REFERENCE AND QUESTIONS

Keep this beside your editor.

These are exact field names from the current Python contracts. Defaults fill omitted optional fields; partial excerpts are not safe replacements for full PATCH payloads.

Group	Exact fields / meaning
Identity / version	schema_version, id, revision, source_type, source_file, document_id, input_method
Document context	company, role, name, email, sections
Content arrays	skills, requirement_groups, education, projects, experience, certifications, internships, hackathons, achievements
Lifecycle / processing	review_status, extraction_mode, parser_version, model_version, prompt_version, llm_status, llm_metadata, warnings
Each skill: identity	skill_id, skill_name, normalized_name, category_code, skill_family
Each skill: evidence	evidence, confidence, source_page, source_start, source_end, source_file, source_document_id, source_type, evidence_type
Each skill: state	verification_status=unverified; claim_status=claimed|needs_review; claim_basis=explicit|inferred
Each requirement group	requirement_id, skill_ids, operator=AND|OR, requirement_type=mandatory|preferred|unspecified, requires_review, evidence, source_page

Questions you should answer without memorizing code

Does Python in a CV mean Python level 8? No. It is an unverified source claim. Numeric self-ratings are collected separately by Profile Builder.

Can it read every CV layout? No. Common text-based PDFs/DOCX work with review; scans, complex reading order and implicit meaning have limits.

What does source evidence prove? Only that the quoted text occurs in the document, not skill proficiency or authorship.

Source of truth and current limits

Read your module's README.md, PLAN.md, CONTRACT.md and USAGE.md; shared/contracts/models.py; docs/BUILD_PLAN.md; docs/DEPLOYMENT_SETUP.md; docs/LLM_EXTRACTION_IMPLEMENTATION.md; verification/RESULTS.md; and docs/TEAM_PRESENTATION_CHECKLIST.md. Full current validated synthetic JSON snapshots are under data/synthetic/handoff/. Snapshot IDs are not live IDs.

Original Participant Brief: pp.7-10 (roles/contracts), p.11 (official test pack), p.13 (stack/done). Event Plan p.4: 30 minutes presentation + 15 Q&A. Original baseline suite (see new checkout results in verification/RESULTS.md): 134 backend, 17 frontend and 14 browser tests. Live Groq quality, cloud RLS/deployment and official organizer-pack/benchmark acceptance remain pending. All guide examples are synthetic.